

COMPUTER ARCHITECTURE

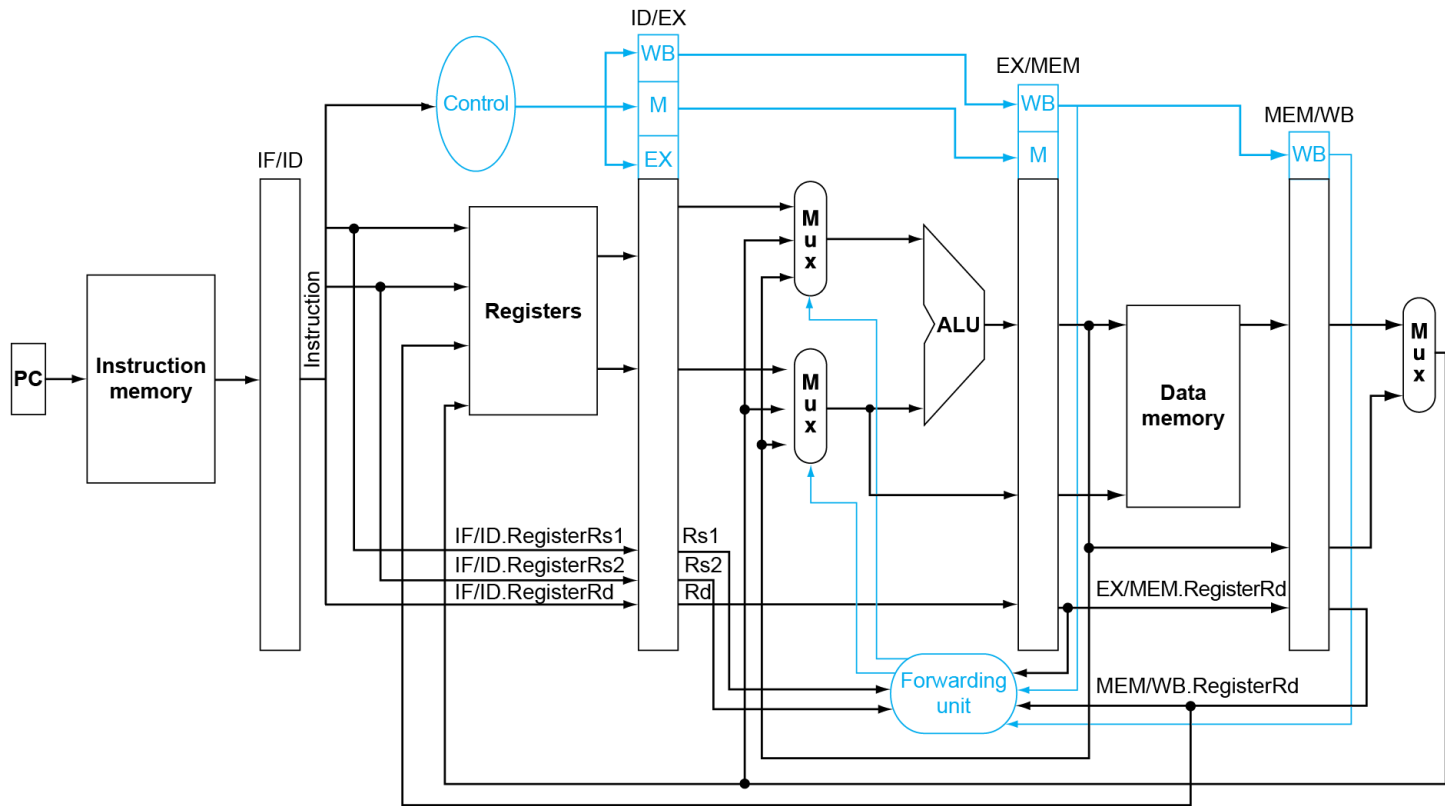


LECTURE 7: Pipelined Processor (cont.)

CONTENTS

- > Data hazard: Load-use
 - Cause
 - Solutions
- > Control hazards
 - Overview
 - Resolution by flushing
 - Delayed decision
 - Branch predictors
 - Branch Target Buffer (BTB)

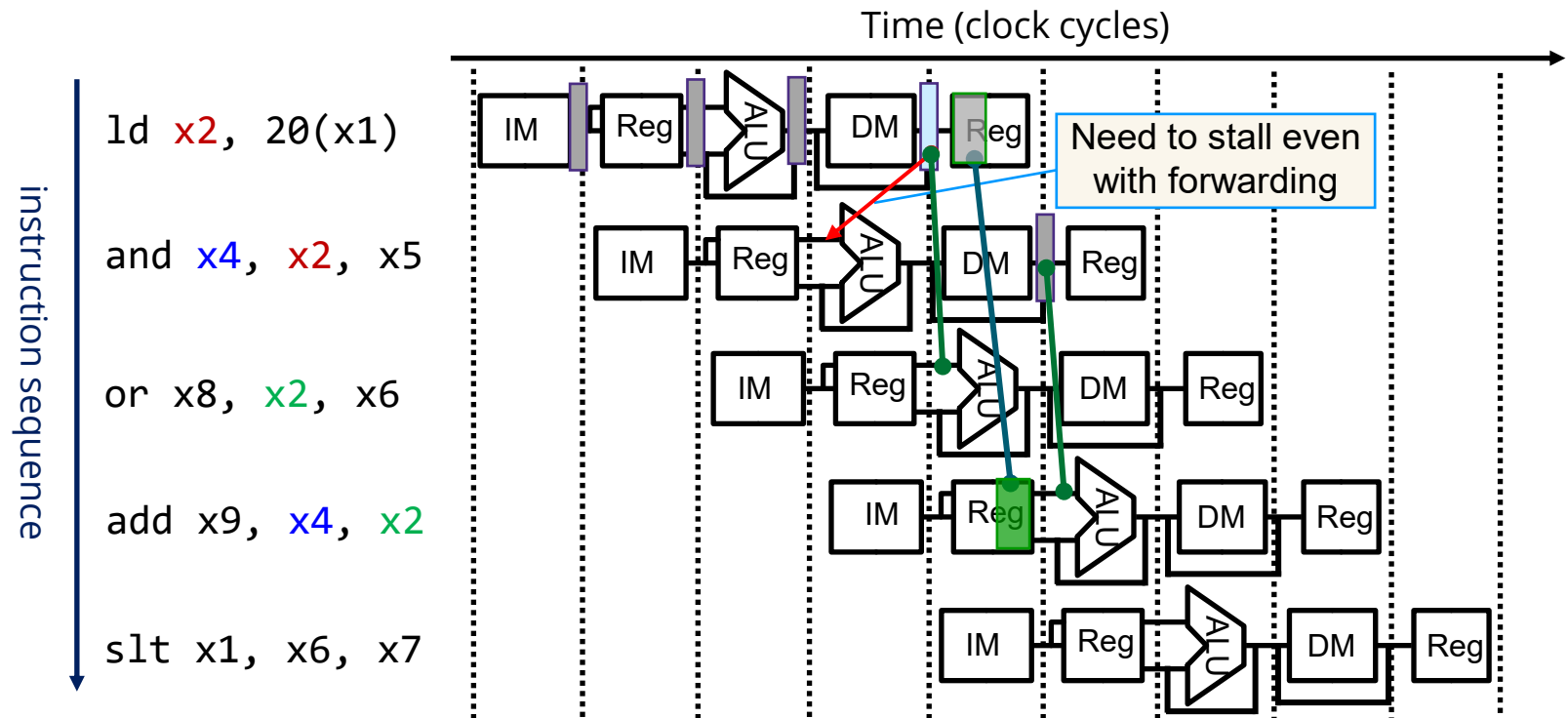
Recap: Datapath with Forwarding



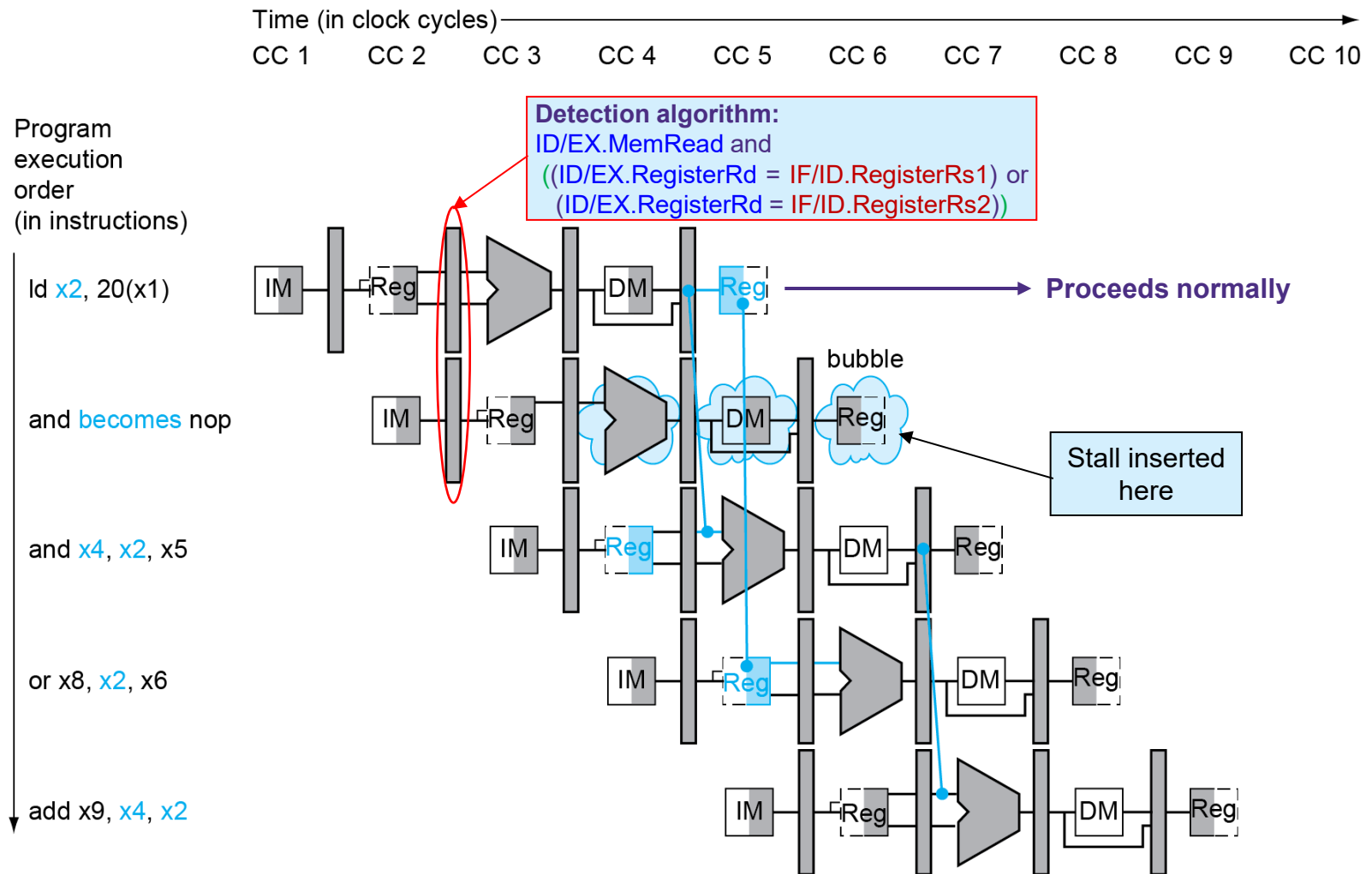
> Does forwarding solve all data hazards?

Load-Use Hazard

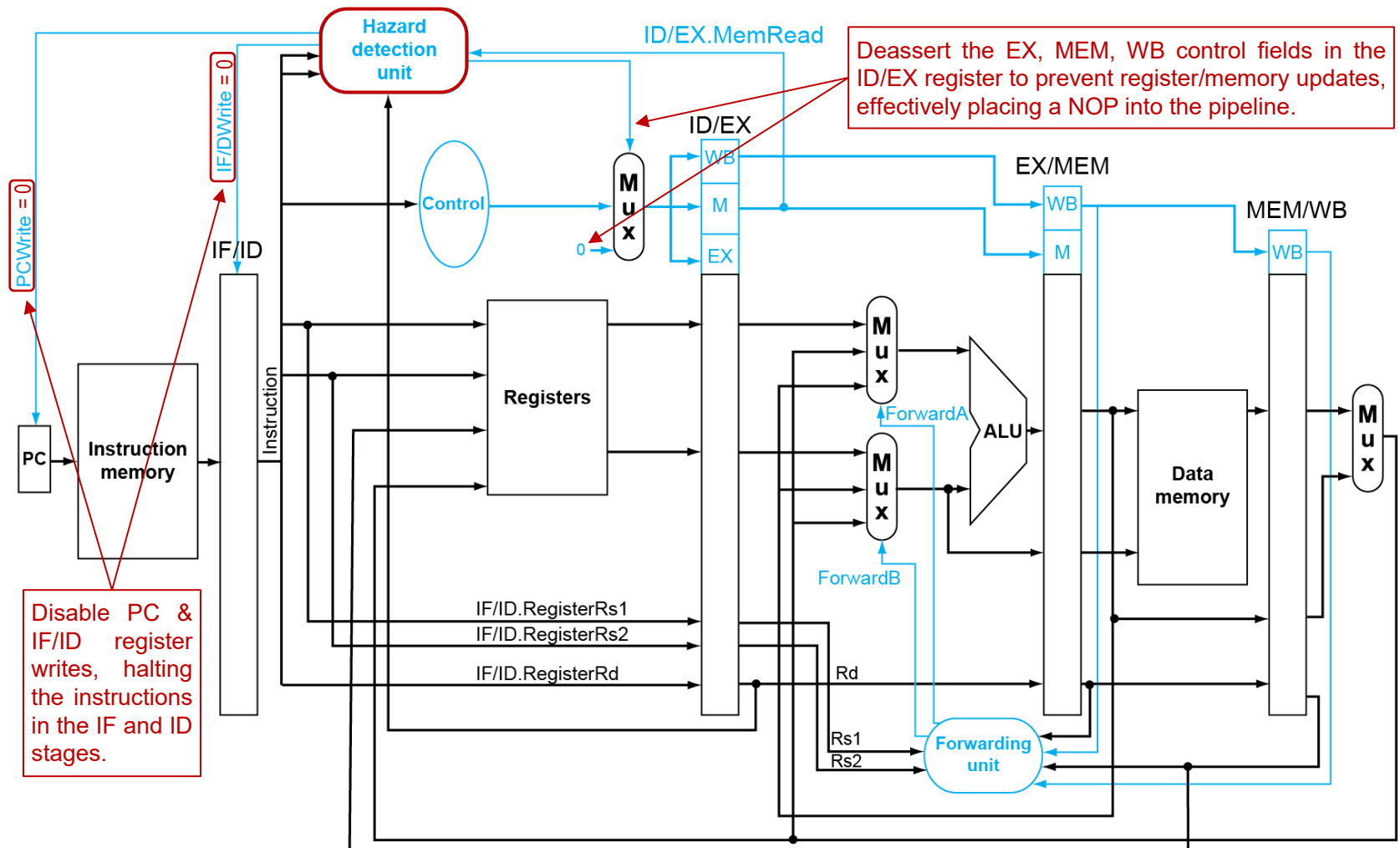
- > Load has a delay that cannot be eliminated by forwarding
 - when an instruction tries to use a register right after it is loaded from memory



Resolving Load-Use Hazard



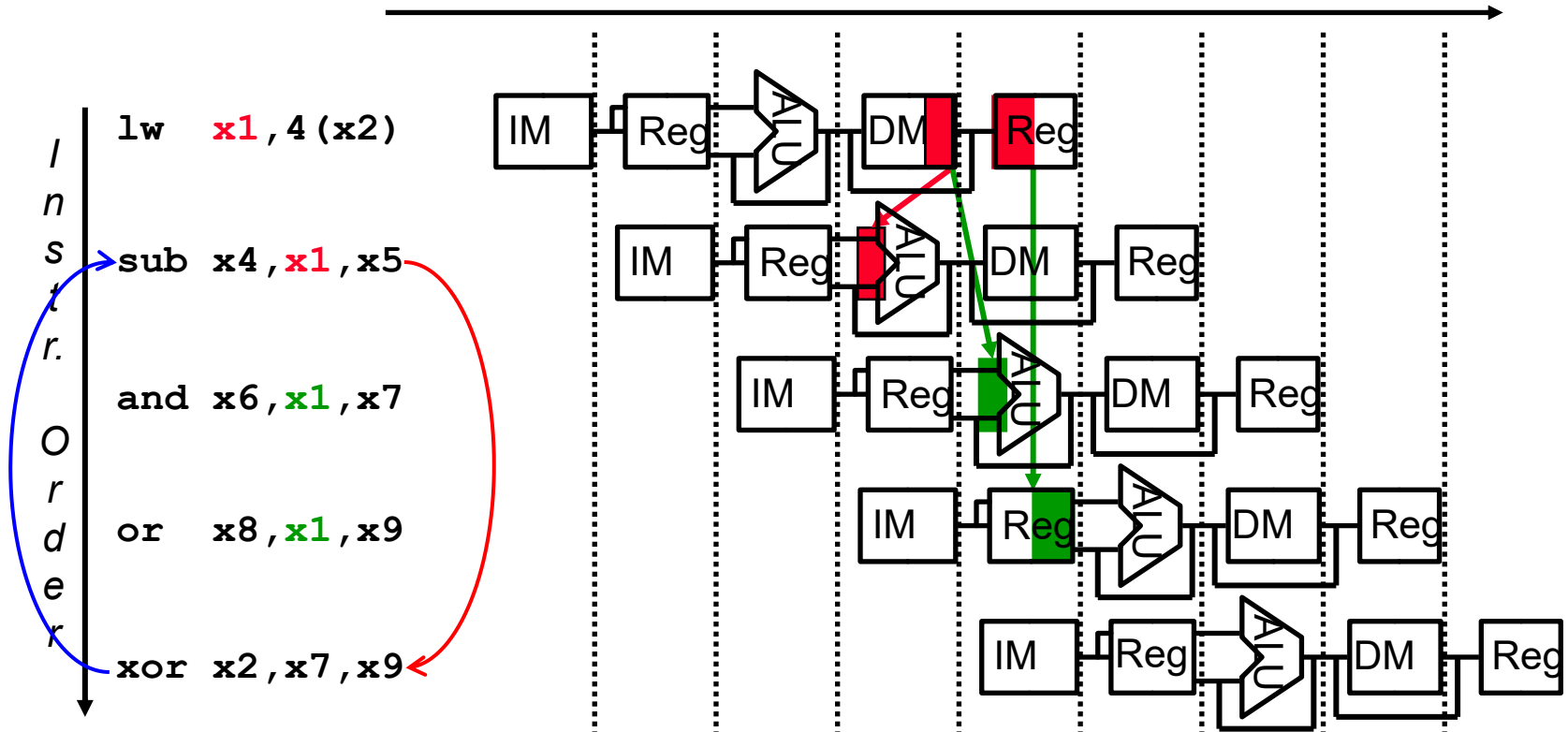
Revised Datapath



Convert a Stall Bubble into Useful Work

- > The instruction slot following a load is called the **load delay slot**
 - Inserting a **nop** in this slot avoids the load-use hazard but wastes one cycle.
- > Improvement: Reorder instructions so that
 - the dependent instruction is moved out of the load delay slot, and
 - an independent instruction is moved into the slot to fill it.
 - ✓ No performance loss!
- > Two approaches to instruction reordering
 - **Static scheduling**: the compiler reorders instructions *at compile time*.
 - **Dynamic scheduling**: the HW reorders instructions automatically *at runtime*.
- > Caveats
 - ✓ Works well when independent instructions are available nearby.
 - ✗ If no such instruction exists → the CPU must **insert a stall**.

Example: Compiler Scheduling

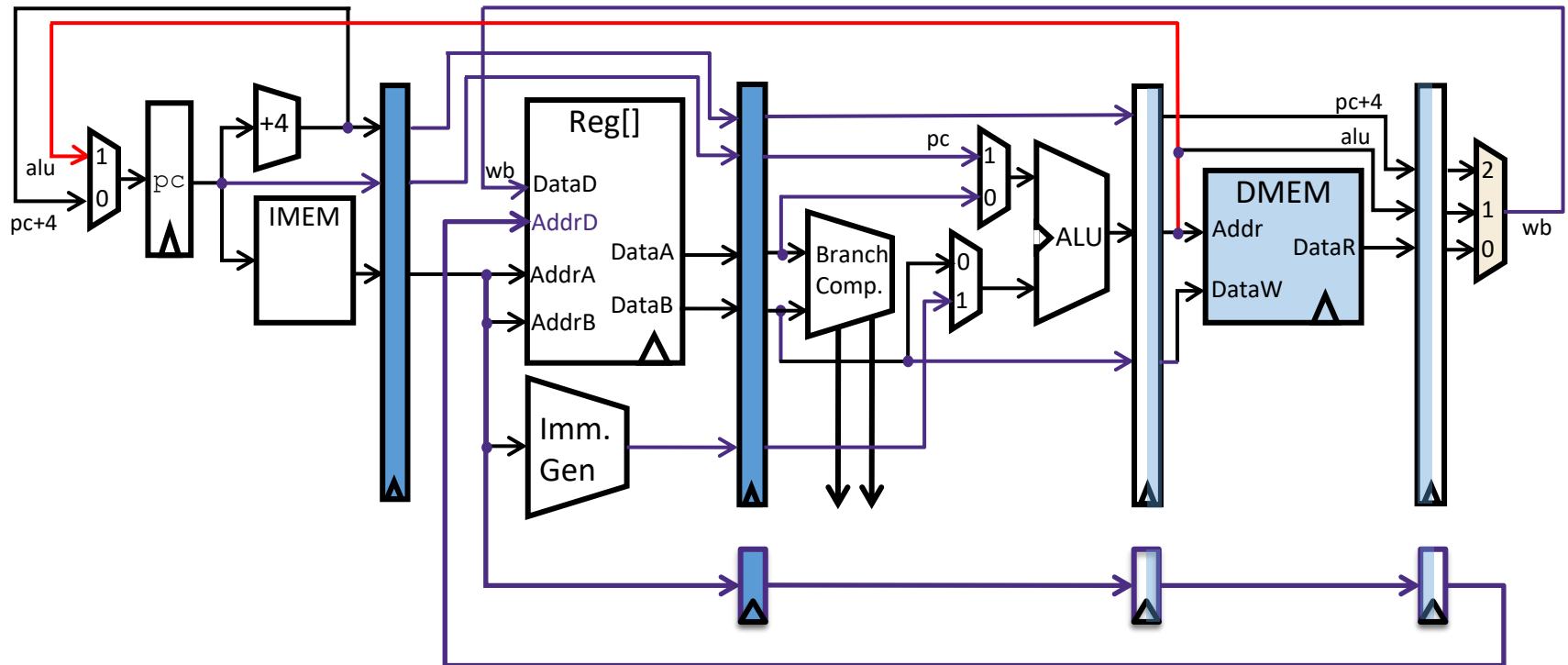


✓ With code scheduling:
5 cycles

⚠ Simple compilation:
6 cycles for 5 instructions

An Issue with Branches

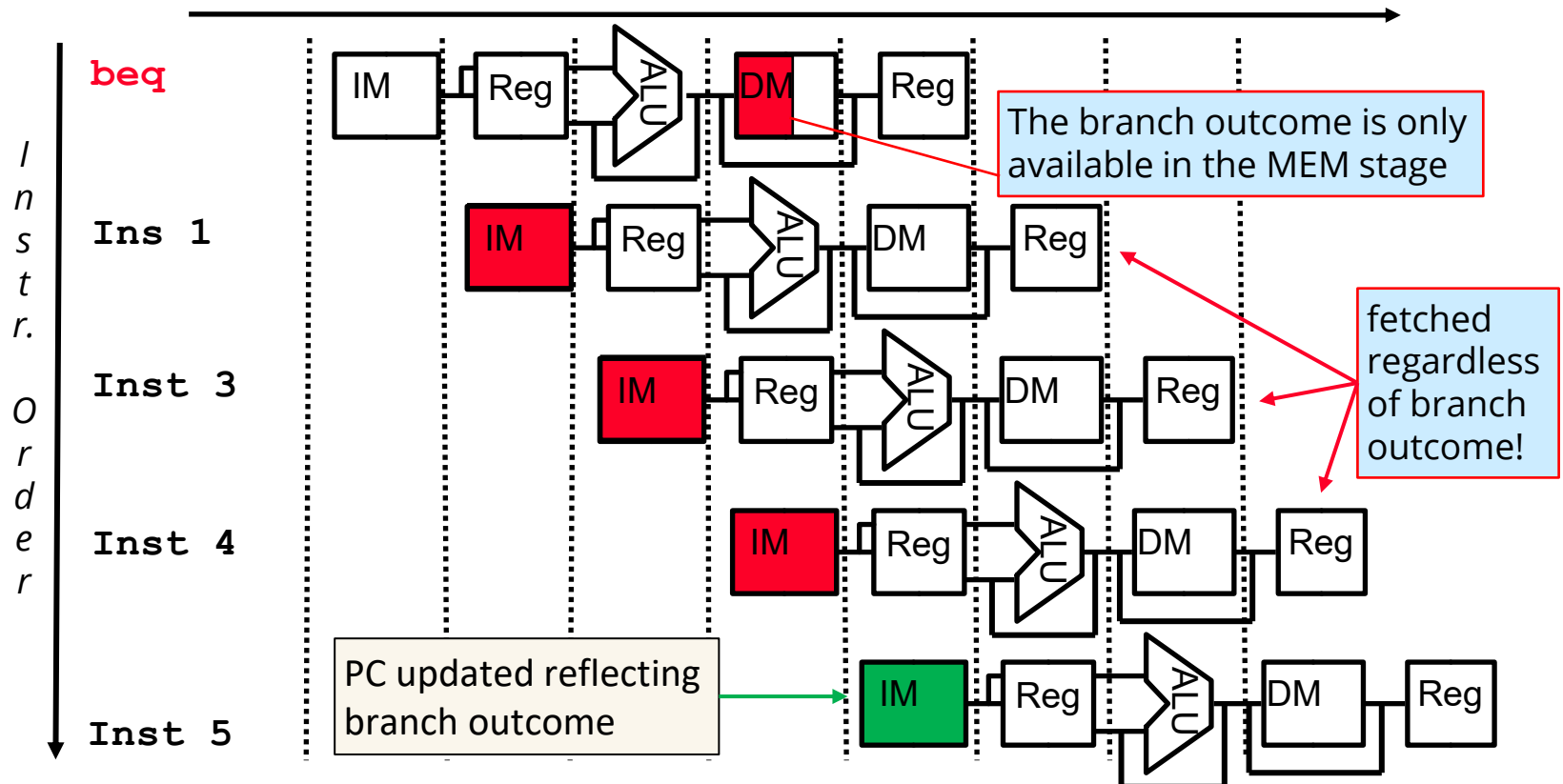
> Assuming a branch instruction (**beq**, **bne**, ...) is in the ID stage



- How can a pipeline fetch the next instruction?

Overview of Control Hazards

- > Occur when the flow of instruction addresses is **not** sequential
 - E.g., if the branch is taken → dependencies backward in time cause hazards

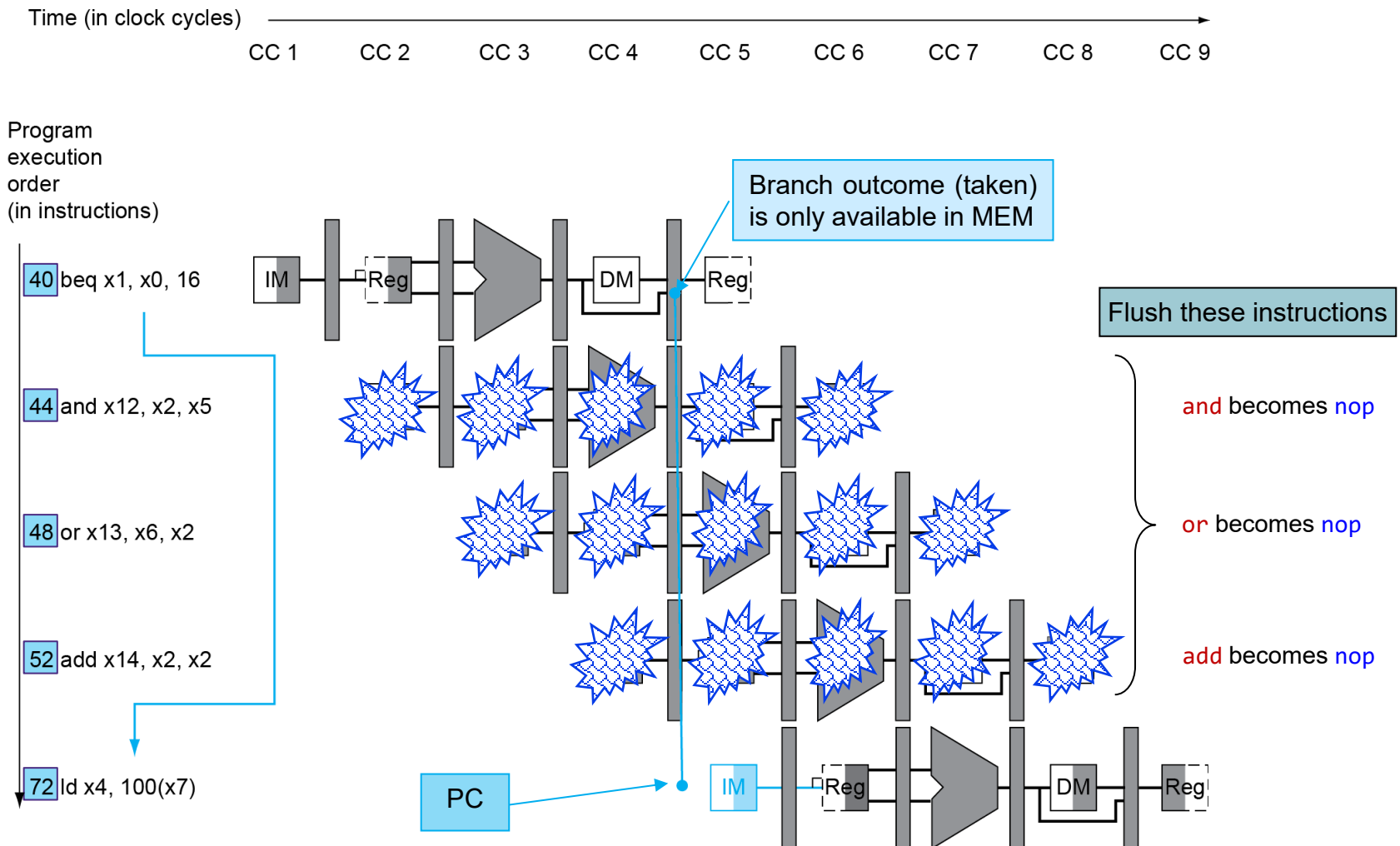


Handling Control Hazards

- > Flushing (stalling): Stop fetching until the branch outcome is known
 - Resolves the hazard, but increases CPI
 - Improvement: Move decision point earlier in the pipeline to reduce stall cycles
- > Delay decision: Move a useful instruction after the branch
 - Requires compiler support
 - Useful instructions may not always exist
- > Predict the branch *outcome* and hope for the best
 - Static prediction
 - Dynamic prediction
- > Branch *target* speculation

There's **nothing** as effective against control hazards as forwarding is for data hazards!

Fixing Branch Hazards by Stalling



Flushing vs. Stalling

> Stalling

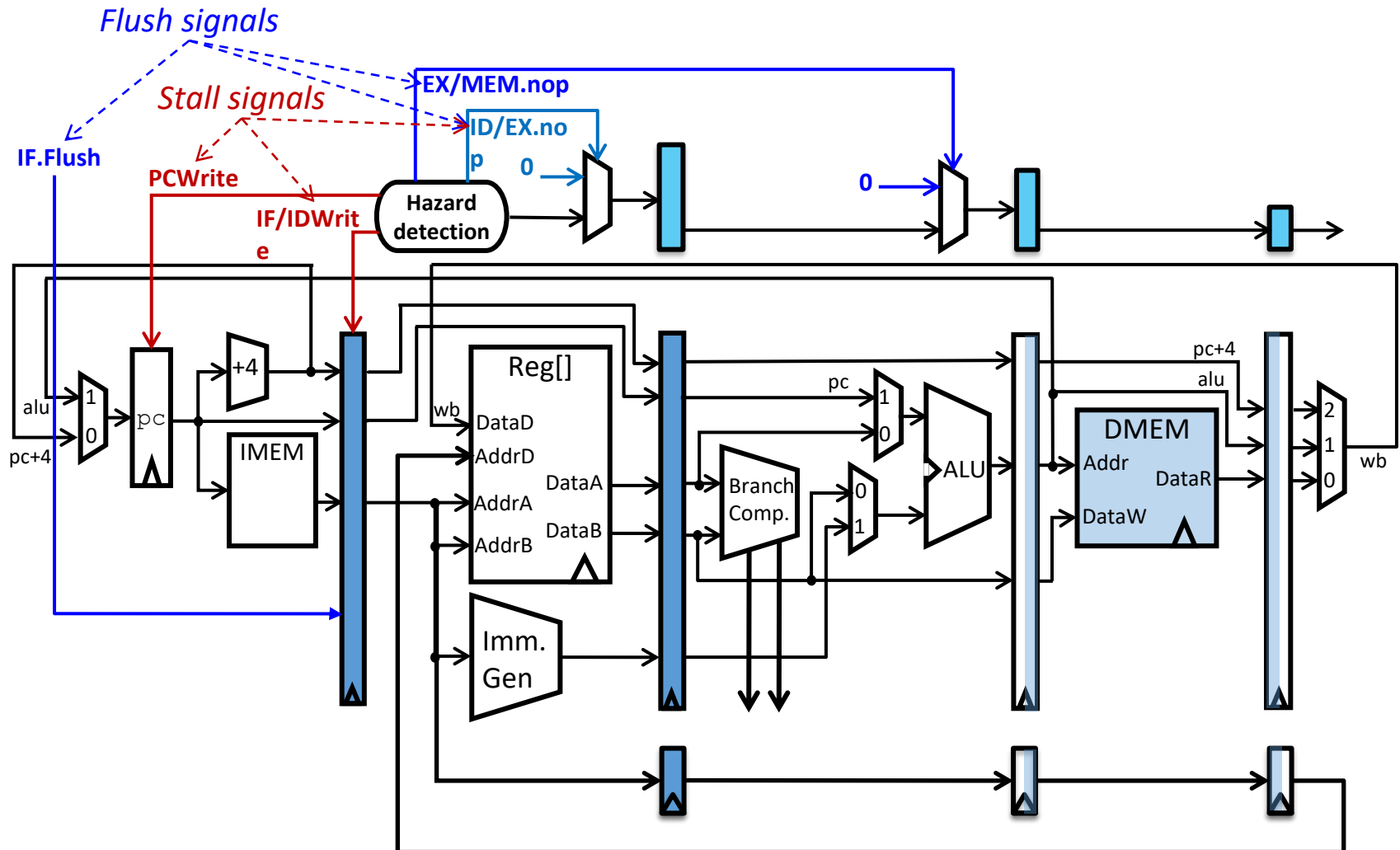
- Inserts a **nop** between load and its dependent instruction
- Prevents earlier pipeline stages from advancing for one cycle
- Later stages continue executing normally
- Implemented by zeroing ID/EX control bits and disabling PCWrite, IF/IDWrite

> Flushing

- Replaces instructions fetched after a branch with **nops**
- The branch proceeds normally, but incorrectly fetched instructions are discarded
- Achieved by asserting an IF.Flush signal to clear IF/ID and resetting ID/EX, EX/MEM control bits

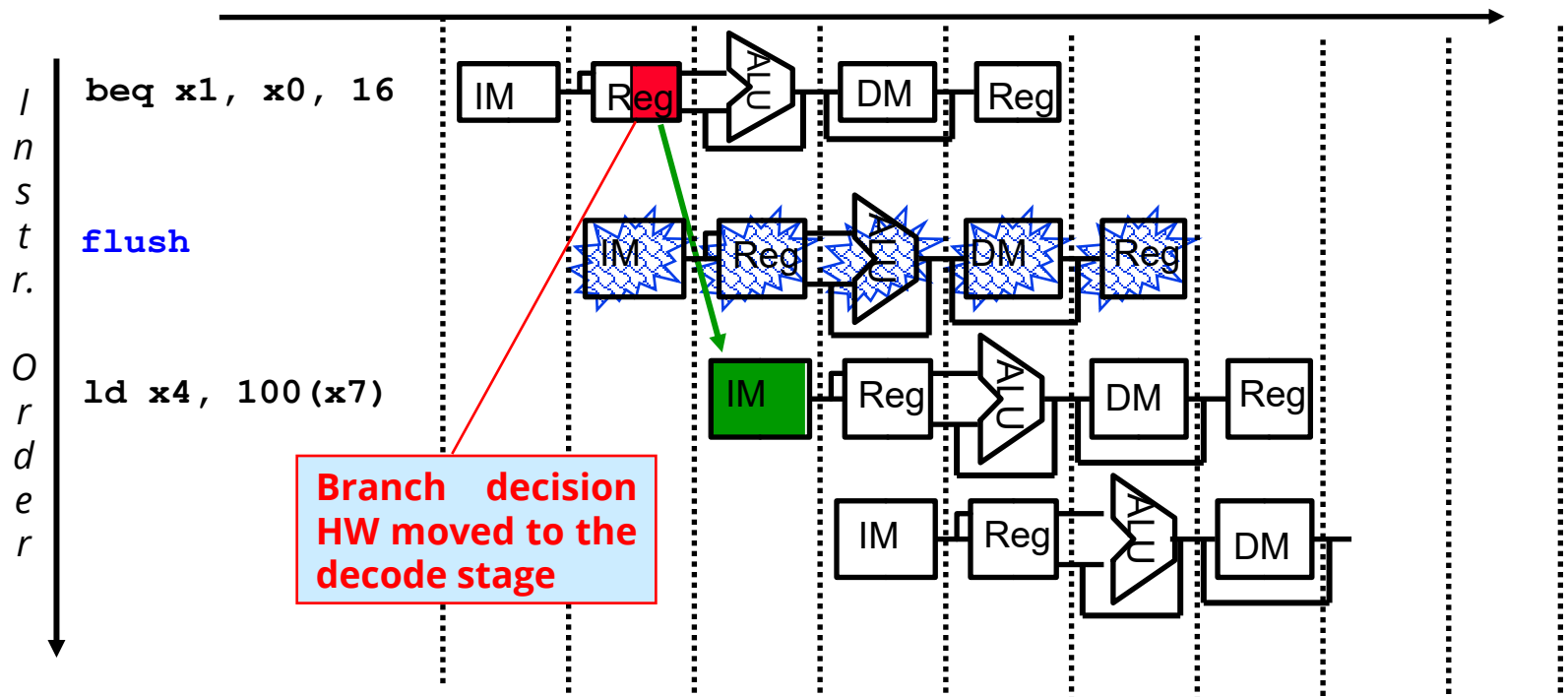
> Both affect CPI

Implementing Stalling and Flushing

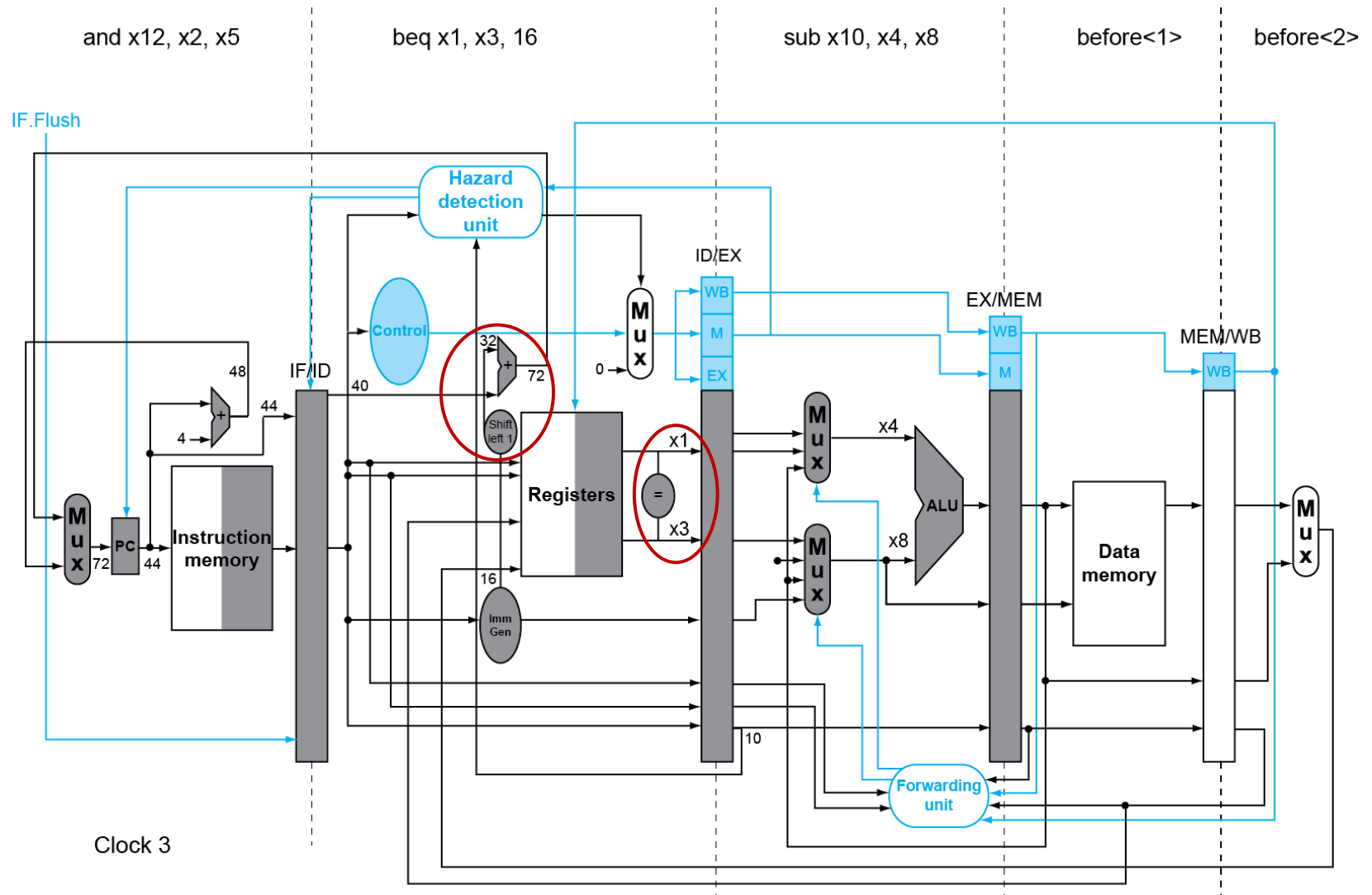


Reducing Branch Delay

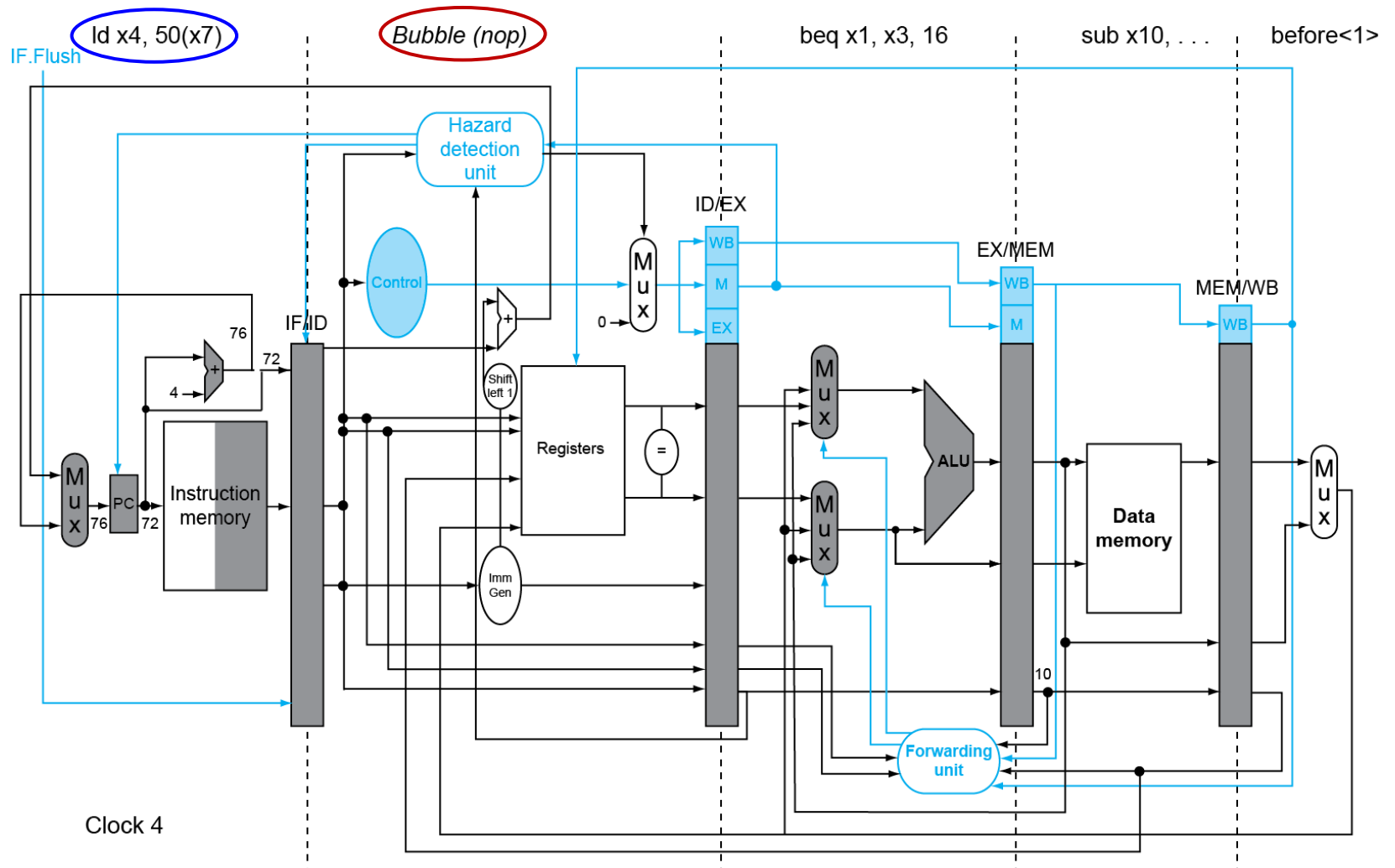
- > Move branch decision back to as early in the pipeline as possible
 - The earliest decision point is in ID stage → **at most 1 cycle** delay.
 - Need extra HW to test registers, calculate branch address, & update PC register



Revised Datapath: Example



Revised Datapath: Example (cont.)



Fixing Branch Hazards by Delayed Decision

branch instruction

branch delay slot (next instruction)

branch target (if branch taken)

- > A one-cycle branch delay introduces one delay slot after the branch
 - The compiler schedules a useful instruction into that slot to avoid a branch stall
 - If no independent instruction is found, compiler fills delay slot with a **nop**
 - This optimization is done at compile time (static scheduling).
- > Limitations in modern processors
 - Pipelines are now deeper & support multiple instruction issue → longer delays
 - A single delay slot is no longer sufficient
 - Delayed branching has largely been replaced by dynamic branch prediction
 - Albeit at a higher hardware cost

Static Prediction

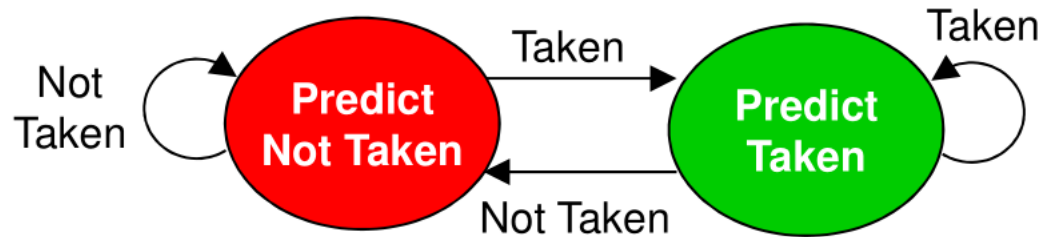
> Key idea:

- Fix the branch outcome at compile time & take corrective action if mispredicted
- Common static strategies:
 - Always not taken → fetch sequentially.
 - Always taken → assume all branches will jump; prefetch from target.
- **Example:** Assume branches are always *not taken*
 - If the branch is taken → flush the next instruction (wasted 1 cycle)
 - If the branch is not taken → no stall, execution continues normally.

> Performance implications:

- Works reasonably well in simple, short pipelines
- In deeper or superscalar pipelines, branch penalties become larger
 - More pipeline stages → longer delay before the branch outcome is known
 - Multiple instruction issue → more instructions are flushed if mispredicted.

1-bit Dynamic Branch Predictor

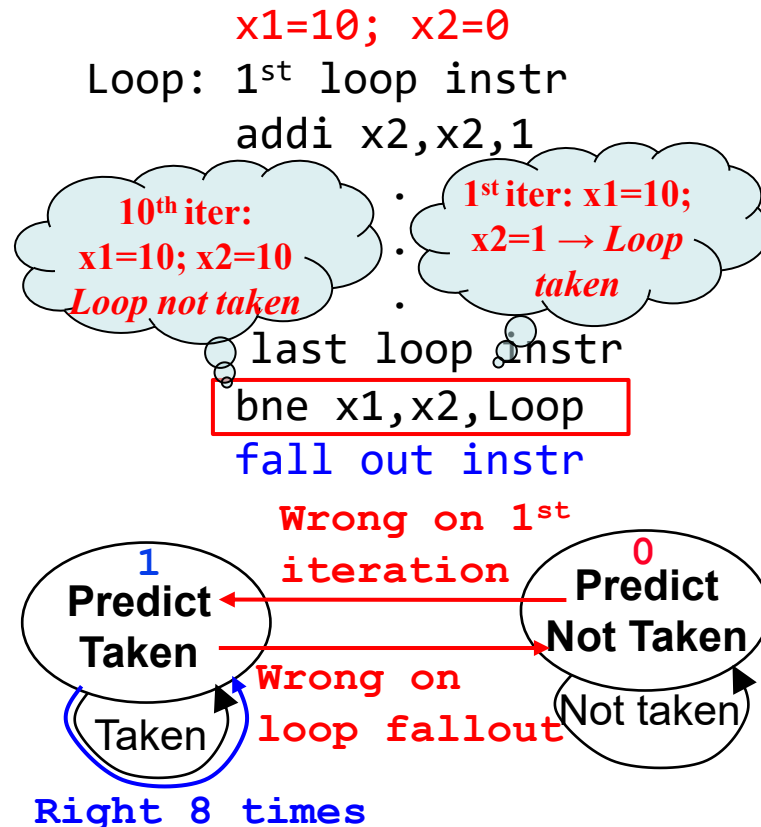


- > Prediction the outcome at runtime using a 1-bit ***saturation counter***
 - Use only 1 bit per branch instruction → the outcome is either taken or not taken
- > Simple to implement
 - Record the last outcome of a branch instruction (Taken/Not taken)
 - Use last outcome to predict future behavior of a branch

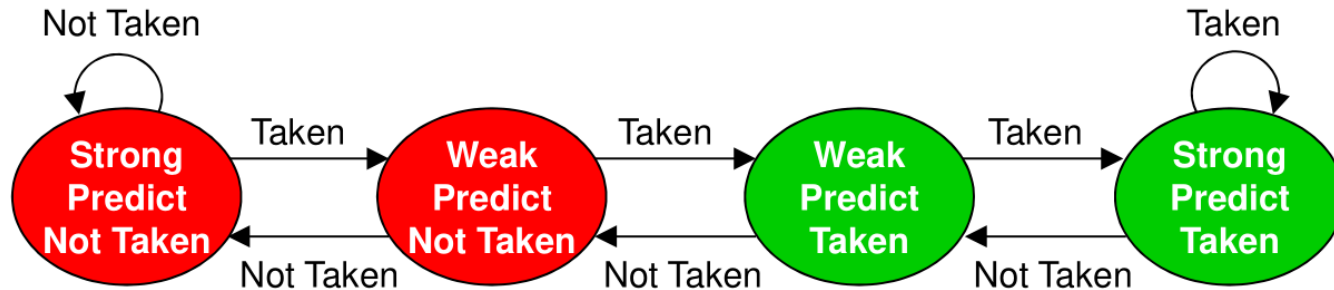
Drawbacks of the 1-bit Predictor

> A 1-bit predictor can be incorrect twice when not taken

- `predict_bit` starts at 0
- 80% accuracy
- Can we do better?



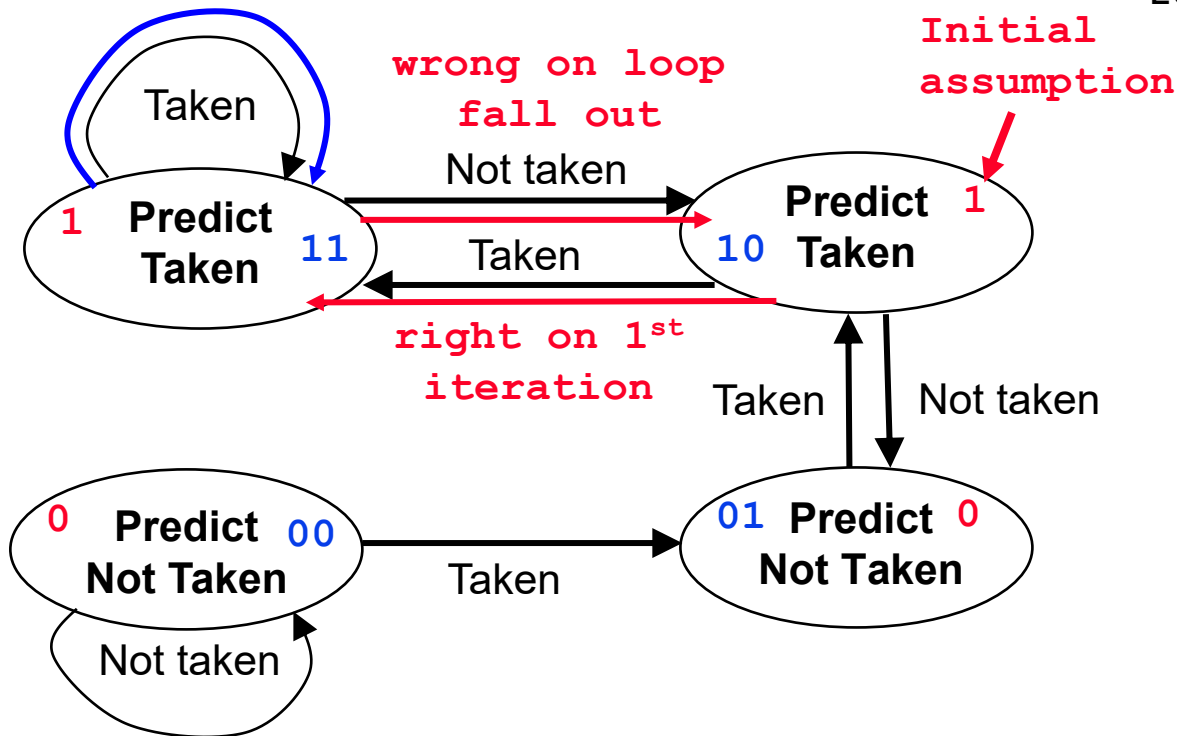
2-bit Dynamic Branch Predictor



- > 2-bit predictor works better than its 1-bit counterpart
 - 1-bit prediction scheme is rarely used in practice
 - 4 states: strong and weak predict taken/predict not taken
 - Must be wrong twice before the prediction bit is changed
- > Implemented as a 2-bit saturating counter
 - Counter is incremented to max=3 when branch outcome is taken
 - Counter is decremented to min=0 when branch is not taken

2-bit Predictor: Example

right 9 times



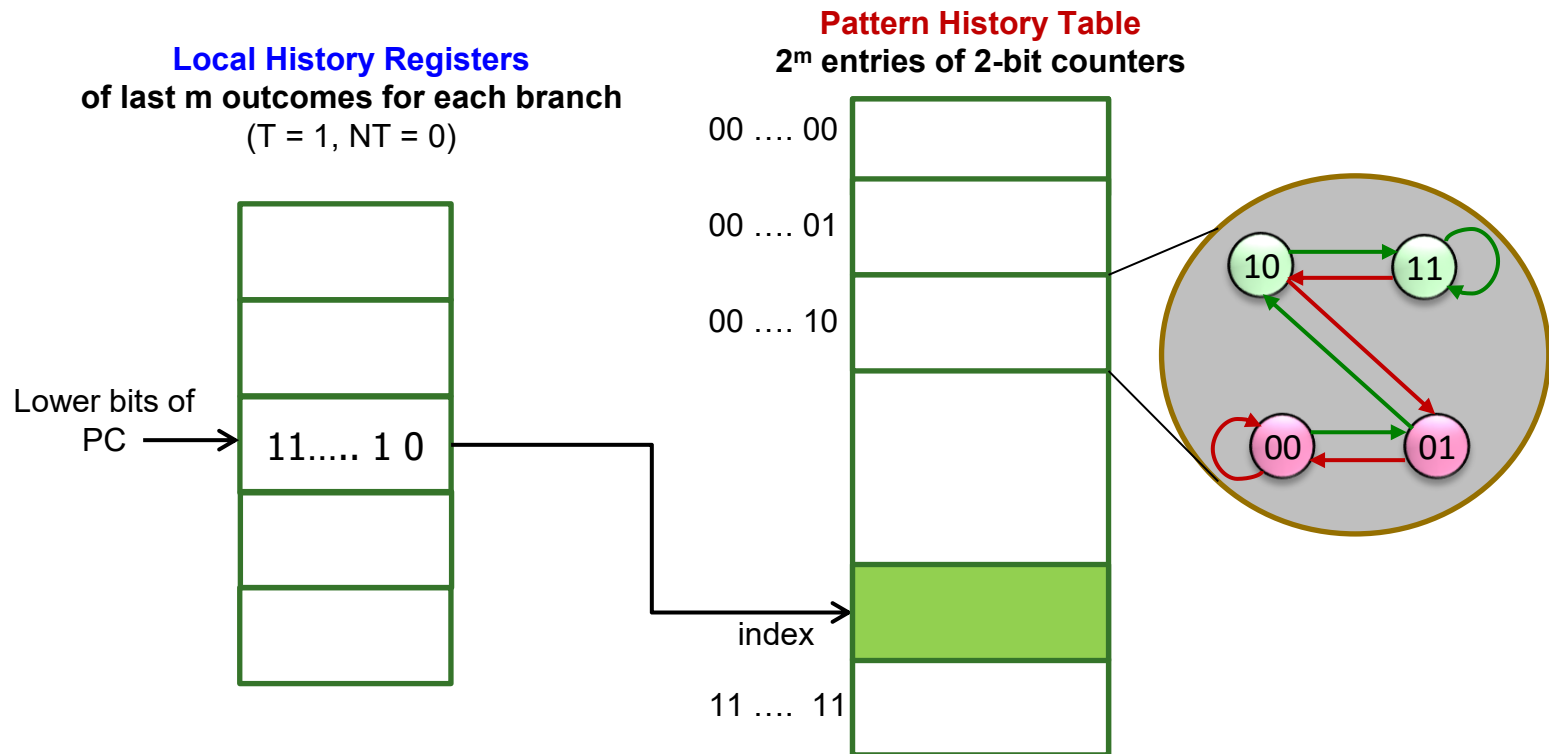
Loop: 1st loop instr
2nd loop instr
.
.
.
last loop instr
bne x1,x2,Loop
fall out instr

> Achieves 90% accuracy

Local Branch Predictor

- > Saturating counters exploit “last time” predictability
 - Limited history depth
 - Cannot learn patterns (e.g., alternating taken/not-taken)
 - **Local branch correlator:** Tracks each branch’s past outcomes to learn its characteristic taken/not-taken pattern.
- > Mechanism:
 - Record recent outcomes in a Local History Registers (LHR) for each branch
 - Select the LHR based on the PC of the branch & use the value stored in the selected LHR as an index into a Pattern History Table (PHT) of 2-bit counters
 - The counter’s state determines the predicted outcome.
- > Advantages and limitations
 - ✓ Captures repetitive behavior (e.g., loops that take the branch 9 times, then not)
 - ✗ More hardware (one LHR per branch + PHT)

Implementation of Local Branch Predictor



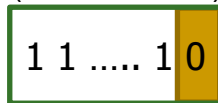
Global Branch Correlator

- > The local predictor doesn't capture correlations across branches
 - Predict based on the branch-specific patterns (local correlations)
 - A branch's outcome can be correlated with other branches' outcomes
 - Example 1: `if (cond1) ...`
`if (cond1 AND cond2) ...`
 - Example 2: `if (cond1) a = 2; ...`
`if (a == 0) ...`
- > Global branch correlator: Using correlation between different branches to predict the next branch behavior
 - Record recent branch outcomes in a Global History Register (GHR)
 - Use the GHR value as an index into a PHT of 2-bit counters
 - The counter's state determines the predicted outcome

Implementation of Global Branch Predictor

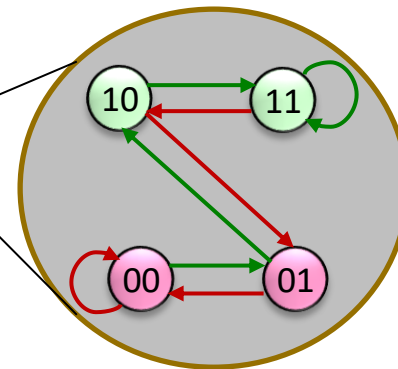
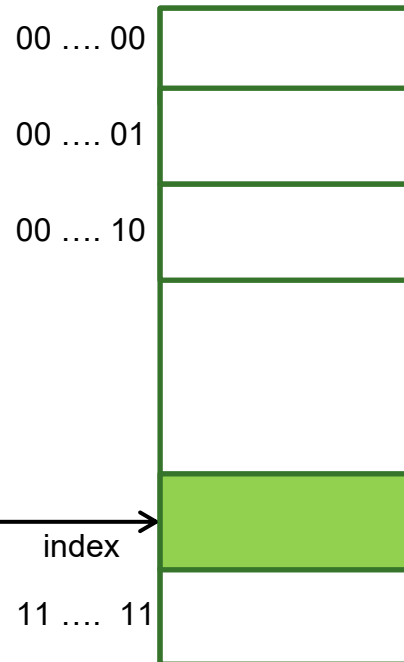
Global history register
of last m branches executed

($T = 1$, $NT = 0$)



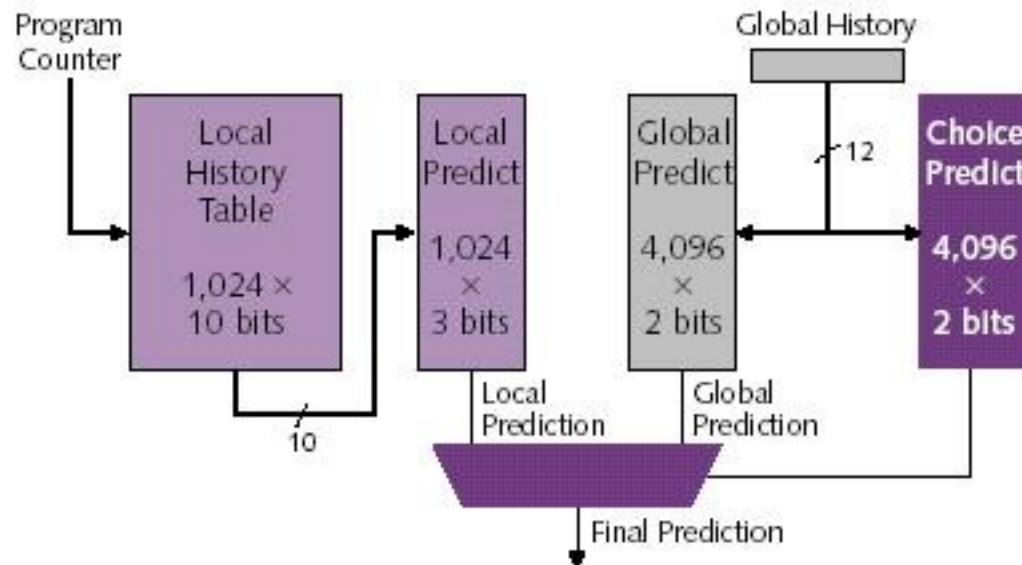
The last
branch's
outcome

Pattern History Table
 2^m entries of 2-bit counters



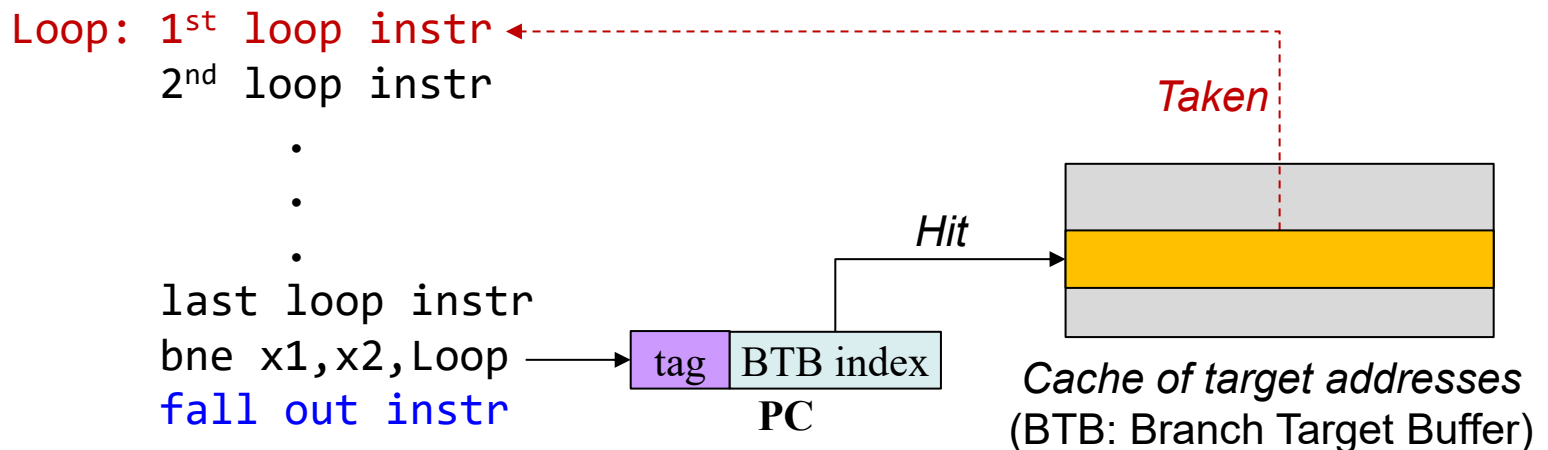
Tournament Predictor

- > Adaptively combining local & global predictors
 - Two predictors make separate predictions
 - A selector table tracks which predictor was more accurate in the past
 - The selector's 2-bit counter decides whose prediction to trust this time
 - If the chosen predictor was correct, the selector strengthens that preference, otherwise, it shifts toward the other predictor.

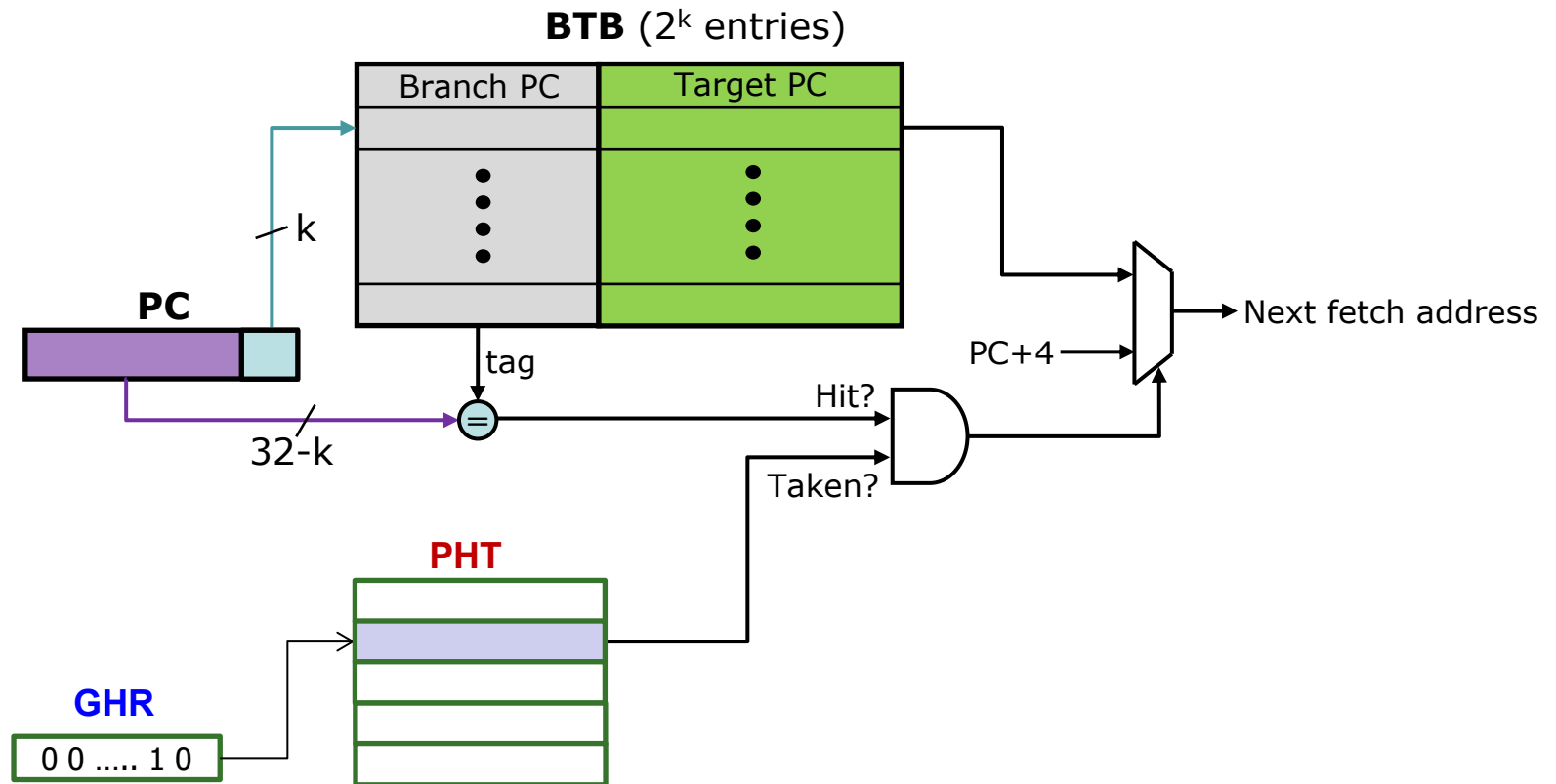


Branch Target Speculation

- > When a branch is predicted “taken”, it’ll need the target address
 - Calculating the target address: 1-cycle penalty for in a 5-stage pipeline
 - Conditional direct branch: constant target address across dynamic instances
- > **Idea:** predict the next instruction from the past target addresses
 - Use a cache, indexed by a portion of the PC, to store the target address from previous instances → **zero** penalty if there’s a hit & the branch is “taken”.



Global Branch Predictor with BTB



NEXT LECTURE

[Flipped class] Midterm Exam

- > Pre-class
 - Review previous lecture
- > In class
 - Exam
 - Solution & discussion
- > Post class
 - Prepare for the next lecture